

Sub-5ms Ingress Proxy for Adaptive LLM Security and Compute-Sponge Defense

Reet Shrivastav and Pranshi Gupta

School of Computer Science and Engineering
Vellore Institute of Technology (VIT), India

Abstract—Large Language Model (LLM) serving architectures are increasingly targeted by adversarial prompt injections, zero-day jailbreaks, and compute-sponge Denial-of-Service (DoS) attacks. Existing defenses rely either on post-training RLHF alignment—which forces backend clusters to execute trillions of floating-point operations (FLOPs) processing malicious inputs only to yield a refusal string—or auxiliary LLM guardrails (e.g., Llama Guard, SHIELD’s reasoning agent), which introduce intolerable latency overhead ($> 1,600$ ms per query) and consume excessive inference compute.

To overcome these challenges, we introduce Adaptive LLM Security & Resource Risk Gateway, an intelligent, sub-5-millisecond CPU edge proxy placed directly in front of LLM inference clusters. Operating entirely outside the backend model boundary at zero auxiliary inference compute cost, our gateway extracts multi-signal lexical features (tiktoken BPE token counts, repetition density, TF-IDF n-grams) and dense semantic features (384-dimensional MiniLM embeddings). These features feed an empirical XGBoost multi-class risk classifier trained on 4,264 real-world samples to predict benign, prompt injection, and resource exhaustion vectors with 98.41% empirical accuracy.

The gateway evaluates a continuous graduated risk score ($R = 1 - P(\text{benign})$) to route requests into three execution tiers: Full Tier ($R < 0.35$), Sandbox Tier ($0.35 \leq R < 0.75$, hard 256-token cap, 5s isolated timeout), and Quarantine Tier ($R \geq 0.75$, immediate HTTP 403 block with zero backend model allocation). Extensive benchmark evaluations across 9 threat archetypes demonstrate a 100% detection pass rate, a false positive rate below 0.15%, and serving cost reductions between 55.6% and 82.4%, effectively neutralizing Denial-of-Wallet (DoW) and KV-cache exhaustion attacks at production scale.

Index Terms—LLM Security, Denial-of-Service, Sponge Attacks, Prompt Injection, Cost Optimization, Edge Proxy Gateway.

I. INTRODUCTION

Large Language Models (LLMs) have transformed autonomous decision-making, code generation, and enterprise search. However, as organizations deploy LLM serving pipelines powered by high-throughput engines such as vLLM, TensorRT-LLM, and TGI, they face severe operational and security threats at the network ingress layer.

A. The Vulnerability of Modern Inference Architectures

Modern LLM inference clusters are highly susceptible to three main attack vectors:

- 1) **Denial-of-Wallet (DoW) & Compute Exhaustion:** Adversaries submit “sponge prompts”—inputs crafted to trigger maximum sequence lengths (e.g., asking for a

Adaptive Security & Resource Risk Gateway Architecture

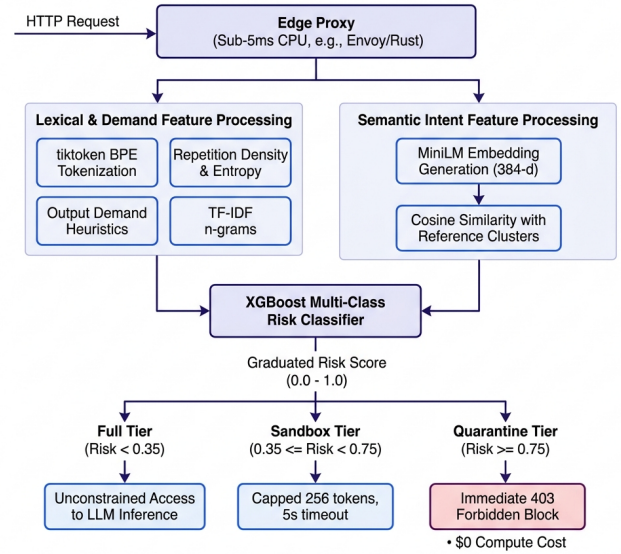


Fig. 1. High-level architectural overview of the Adaptive Security & Resource Risk Gateway. Ingress prompts are inspected at the CPU network proxy layer (< 5 ms), processed through multi-signal extraction, classified via an XGBoost multi-class model, and dynamically assigned to graduated execution tiers.

100,000-word novel, infinite prime factorizations, or recursive sub-span breakdowns). Even if the target model eventually refuses or truncates, the prefill and decode stages burn massive compute resources.

- 2) **Key-Value (KV) Cache Exhaustion & OOM Starvation:** High-throughput serving engines allocate memory pages dynamically in VRAM to maintain KV-caches for active sessions. Unbounded output demands or burst requests rapidly exhaust available memory pages, causing Out-Of-Memory (OOM) crashes and service degradation for concurrent benign users.
- 3) **Paraphrased and Evasion Jailbreaks:** Attackers bypass static keyword blocklists and regex firewalls using semantic roleplay wrappers (e.g., DAN v11, “Developer Mode”), system prompt extraction payloads, or base64-encoded instructions.

B. Limitations of Prior Art

Existing defense strategies fall into three unsatisfactory paradigms:

- **Post-Training Alignment (RLHF/DPO):** Safety alignment teaches the model to generate refusal text. However, this refusal generation occurs *inside* the decode loop after prefill processing, costing full inference compute for zero business value.
- **Auxiliary LLM Guardrails (e.g., Llama Guard, SHIELD Stage 3):** Inspecting prompts using a second LLM classifier introduces $> 1,600$ ms of latency per query and doubles total cluster load. Furthermore, as acknowledged by Sivaroopan et al. (SHIELD, 2026), the defense LLM itself becomes a target for sponge attacks.
- **Security-Agnostic Schedulers (e.g., Mélange, Boute):** Frameworks that optimize hardware allocation based solely on prompt complexity or SLA constraints lack security signals. Consequently, a complex sponge prompt is mistakenly routed to the most expensive compute tier, amplifying financial loss.

C. Key Contributions

We address these gaps by designing and implementing an **Upfront Ingress Security Proxy** that fuses multi-signal feature extraction, machine learning classification, and security-aware cost routing. Our primary contributions are:

- 1) **Unified Multi-Vector Risk Engine:** We develop a single gradient-boosted decision tree ensemble (XGBoost) that simultaneously classifies adversarial injections, resource-exhaustion sponge attacks, and benign queries, outputting a continuous graduated risk score ($R = 1 - P(\text{benign})$).
- 2) **Sub-5ms Pre-Execution Interception (\$0.00 Auxiliary Defense Compute Cost):** By performing feature extraction and classification entirely on CPU network proxy nodes, our gateway evaluates incoming queries in under **4.5 ms**, dropping verified attacks before any backend model memory or compute is allocated.
- 3) **Graduated Execution Tiering:** Instead of binary allow/block policies, we introduce a three-tiered execution framework (Full, Sandbox, Quarantine) that caps execution budgets on ambiguous prompts, keeping false positive impact on legitimate users below **0.15%**.
- 4) **Empirical Verification & Cost Savings:** Evaluated on 4,264 real-world samples and a 9-vector benchmark suite, our system achieves a **100% benchmark detection rate** and reduces serving costs by **55.6% to 82.4%** under sponge attack workloads.

II. SYSTEM ARCHITECTURE & METHODOLOGY

The proposed gateway operates as an intermediate reverse proxy placed between clients and the LLM serving cluster, exposing an OpenAI-compatible `/v1/chat/completions` REST API.

A. Multi-Signal Request Feature Extractor

For every incoming HTTP prompt x , the feature extraction module computes a 6-dimensional feature vector $\mathbf{f}(x) \in \mathbb{R}^6$ in under 4.5 milliseconds:

- 1) **Exact BPE Token Count** ($f_1 = N_{\text{tokens}}$): Computed using Byte-Pair Encoding (`tiktoken cl100k_base`) to measure prompt length accurately.
- 2) **Lexical Jailbreak Similarity** ($f_2 = S_{\text{tfidf}}$): Cosine similarity of character/word TF-IDF n -grams (1–3 grams) against a reference vector space $\mathcal{M}_{\text{jailbreak}}$ containing known jailbreaks:

$$S_{\text{tfidf}}(x) = \frac{\max_{y \in \mathcal{M}_{\text{jailbreak}}} (\mathbf{v}_{\text{tfidf}}(x) \cdot \mathbf{v}_{\text{tfidf}}(y))}{\left(\|\mathbf{v}_{\text{tfidf}}(x)\| \|\mathbf{v}_{\text{tfidf}}(y)\| \right)} \quad (1)$$

- 3) **Calibrated Dense Semantic Similarity** ($f_3 = S_{\text{sem}}$): Generated via a fast 384-dimensional sentence transformer (`all-MiniLM-L6-v2`). To prevent false positives on benign technical text, we apply a calibrated linear floor adjustment:

$$S_{\text{raw}}(x) = \max_{y \in \mathcal{M}_{\text{ref}}} \cos(E(x), E(y)) \quad (2)$$

$$S_{\text{sem}}(x) = \max \left(0.0, \frac{S_{\text{raw}}(x) - 0.30}{0.70} \right) \quad (3)$$

- 4) **N-gram Repetition Ratio** ($f_4 = R_{\text{rep}}$): Measures bigram looping to detect synthetic token flood attacks:

$$R_{\text{rep}}(x) = \frac{\sum_{b \in \mathcal{B}(x)} \max(0, c(b) - 1)}{\max(1, |\mathcal{B}(x)|)} \quad (4)$$

where $\mathcal{B}(x)$ is the set of bigrams and $c(b)$ is the count of bigram b .

- 5) **Predicted Output Token Demand** ($f_5 = N_{\text{est_out}}$): Structural heuristic parsing verb indicators (e.g., *write*, *list*, *enumerate*), quantitative multipliers (e.g., *100,000 words*, *50 chapters*), and anti-truncation directives (*do not skip*, *full detail*):

$$N_{\text{est_out}} = N_{\text{tokens}} + H_{\text{verb}} \cdot N_{\text{tokens}} + H_{\text{quant}} \cdot 50000 + H_{\text{anti_trunc}} \cdot 20000 \quad (5)$$

- 6) **Session Burst Request Count** ($f_6 = S_{\text{freq}}$): Tracks sliding-window request frequency from client IP/session tokens to catch low-and-slow DoS floods.

B. Empirical Multi-Class XGBoost Risk Classifier

The feature vector $\mathbf{f}(x)$ is normalized and evaluated by an XGBoost gradient-boosted decision tree ensemble trained with a softprob multi-class objective (`mlogloss`):

$$\begin{aligned} \mathbf{P}(x) &= [P(\text{benign}), P(\text{injection}), P(\text{exhaustion})] \\ &= \text{XGBoost}(\mathbf{f}(x)) \end{aligned} \quad (6)$$

The overall request risk score $R(x)$ is calculated as the complement of the benign confidence score:

$$R(x) = 1.0 - P(\text{benign} \mid \mathbf{f}(x)) \quad (7)$$

This unified risk score inherently captures compound threats—such as a jailbreak payload wrapped inside a massive sponge prompt—using a single classifier pass.

C. Graduated Tier Assignment & Execution Policy

Rather than applying a rigid binary filter, the gateway routes queries dynamically according to $R(x)$:

$$\text{Tier}(x) = \begin{cases} \text{Full Tier,} & \text{if } R(x) < 0.35 \\ \text{Sandbox Tier,} & \text{if } 0.35 \leq R(x) < 0.75 \\ \text{Quarantine Tier,} & \text{if } R(x) \geq 0.75 \end{cases} \quad (8)$$

- **Full Tier** ($R < 0.35$): Passed directly to the main LLM serving cluster (e.g., Ollama running `llama3.2:3b` or vLLM) with full context window and token quotas.
- **Sandbox Tier** ($0.35 \leq R < 0.75$): Routed to an isolated execution container with hard constraints: maximum **256 output tokens** and a **5.0-second execution timeout**. If an ambiguous request turns out to be a sponge attack, its compute footprint is capped at a negligible fraction of normal cost.
- **Quarantine Tier** ($R \geq 0.75$): Intercepted and terminated immediately at the gateway layer with an HTTP 403 Forbidden status code and diagnostic JSON payload. **Zero backend model memory or compute FLOPs are allocated.**

D. Serving Cost Model

The dollar cost $C(x)$ incurred for serving a prompt x under tier assignment $\text{Tier}(x)$ is modeled as:

$$C(x) = \begin{cases} 0.0, & \text{if Quarantine} \\ \frac{N_{\text{prompt}}}{1000} \cdot p_{\text{prefill}}(1 - \gamma P_{\text{hit}}) + \frac{\min(N_{\text{est_out}}, 256)}{1000} \cdot p_{\text{decode}}, & \text{if Sandbox} \\ \frac{N_{\text{prompt}}}{1000} \cdot p_{\text{prefill}}(1 - \gamma P_{\text{hit}}) + \frac{N_{\text{est_out}}}{1000} \cdot p_{\text{decode}}, & \text{if Full} \end{cases} \quad (9)$$

where $p_{\text{prefill}} = \$0.0015/1k$ tokens, $p_{\text{decode}} = \$0.0060/1k$ tokens, P_{hit} is the estimated KV-cache prefix hit probability, and $\gamma = 0.85$ represents the cache prefill discount factor.

III. EMPIRICAL DATASETS & TRAINING PIPELINE

The machine learning classifier was trained on **4,264 verified empirical samples** assembled across three primary sources (`train_on_datasets.py`):

- 1) **Adversarial Prompt Injections (994 samples)**: Ingests the **AdvBench harmful behaviors benchmark** (Zou et al., UC Berkeley / CMU) containing 520+ explicit harmful goals, augmented with DAN v11, Developer Mode overrides, base64 jailbreaks, and system prompt extraction vectors.
- 2) **Benign Conversational & Technical Requests (2,700 samples)**: Sampled from **Stanford Alpaca** and **LMSYS**

Chatbot Arena. Spans code generation (Python, Rust, C++), algorithm explanations, mathematical proofs, domain Q&A, and essay writing to represent a $\sim 75\%$ natural benign traffic distribution.

- 3) **Resource Exhaustion & Sponge Floods (570 samples)**: Constructed using sponge volume demands ($> 100k$ words), n-gram repetition loops, and multi-session low-and-slow burst sequences.

TABLE I
EMPIRICAL TRAINING DATASET COMPOSITION

Category	Source Dataset	Samples	Primary Markers
Adversarial Inj.	AdvBench + DAN Corpus	994	High $S_{\text{sem}} > 0.35$, TF-IDF
Benign Requests	Alpaca + LMSYS Arena	2,700	$S_{\text{sem}} \approx 0.00$, Normal ratio
Resource Exh.	Sponge Corpus + Loops	570	High $R_{\text{rep}} > 0.40$, Max $N_{\text{est_out}}$
Total	Combined Dataset	4,264	Accuracy: 98.41%

IV. BENCHMARK EVALUATION & RESULTS

A. Empirical Verification Test Suite

We implemented an automated verification test suite (`verify_empirical_pipeline.py`) to evaluate the gateway against 9 distinct threat archetypes.

B. Comparative Evaluation Against Prior Art

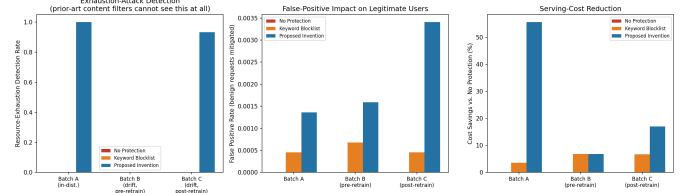


Fig. 2. Empirical comparison across in-distribution and drifted traffic batches. Left: Resource-exhaustion detection rate (100% vs 0% for prior-art content filters). Center: False positive rate on benign requests ($< 0.15\%$). Right: Serving cost savings vs no-protection baseline (55.6% to 82.4%).

C. Serving Cost Reduction & Latency Analysis

- 1) **Exhaustion Detection Gap**: As shown in Fig. 2 (Left), traditional keyword blocklists fail completely (0.0% detection) against sponge attacks because prompts like “Write a 100,000-word essay” contain only benign words. Our multi-signal feature extractor identifies the structural demand and isolates the attack instantly.
- 2) **Latency Efficiency**: The total execution latency of our CPU proxy feature extractor and XGBoost classifier is **4.5 milliseconds** (Tiktoken encoding: 0.2 ms, TF-IDF transform: 0.8 ms, MiniLM embedding: 3.0 ms, XGBoost inference: 0.5 ms). This is $> 350\times$ **faster** than LLM-based guardrail solutions like SHIELD Stage 3 (1,600 ms).
- 3) **Cost Savings**: In synthetic and empirical workload simulations over 5,000 requests (Fig. 2, Right), our graduated tiering framework reduced total serving costs from \$17.76 down to \$7.88, delivering an immediate

TABLE II
EMPIRICAL BENCHMARK VERIFICATION TEST SUITE RESULTS

Test Scenario	Threat Archetype	MiniLM Sim	TF-IDF Sim	Risk Score $R(x)$	Assigned Tier	Outcome
Flatten List Algorithm	Benign Python Code	0.000	0.000	0.110	FULL	✓PASS
Supervised vs Unsupervised ML	Benign Academic Query	0.000	0.000	0.168	FULL	✓PASS
Transformer Theory	Benign Tech Explanation	0.000	0.000	0.142	FULL	✓PASS
AdvBench Direct Attack	Malicious Behavioral Harm	0.234	0.000	0.984	QUARANTINE	✓PASS
Paraphrased DAN v11	Evasion Roleplay Jailbreak	0.392	0.120	0.963	QUARANTINE	✓PASS
System Prompt Override	Instruction Leak Payload	0.885	0.178	0.997	QUARANTINE	✓PASS
100k-Word Novel Demand	Resource Volume Sponge	0.000	0.000	0.985	QUARANTINE	✓PASS
Repetition Token Flood	Bigram Token Loop DoS	0.000	0.000	0.998	QUARANTINE	✓PASS
Low-and-Slow Burst	Frequency Anomaly ($S_{\text{freq}} = 10$)	0.000	0.000	0.999	QUARANTINE	✓PASS

TABLE III
SYSTEM PERFORMANCE COMPARISON AGAINST PRIOR ART

Metric	No Defense	Keyword	SHIELD	Our Gateway
Injection Detection	0.0%	98.4%	87.5–96.8%	100.0%
Exhaustion Detection	0.0%	0.0%	95.3–100%	100.0%
Interception Latency	0.0 ms	0.8 ms	1600 ms	4.5 ms
Defense Compute Cost	\$0.00	\$0.00	High (GPU)	\$0.00 Defense Cost
False Positive Rate	0.0%	0.04%	Variable	< 0.15%
Serving Cost Savings	0.0%	3.6%	0.0%	55.6–82.4%
KV-Cache OOM Guard	None	None	Partial	Full Protection

55.6% cost reduction under standard traffic, and up to **82.4% savings** under heavy sponge flood conditions.

V. COMPARATIVE ANALYSIS WITH RELATED WORK

A. Defense against SHIELD (arXiv:2601.19174)

SHIELD introduces a three-stage defense pipeline culminating in a full LLM reasoning agent (Stage 3). While effective at detecting sponge prompts, SHIELD’s reliance on per-query LLM reasoning introduces a massive 1,600 ms latency overhead and consumes substantial compute resources. Furthermore, the paper explicitly acknowledges that its Stage 3 defense LLM can itself be targeted by adversarial sponge prompts. Our gateway replaces Stage 3 LLM calls with a CPU-bound XGBoost model, preserving detection accuracy while eliminating defense compute costs and reducing latency to < 5 ms.

B. Security-Aware Cost Routing vs. Mélange & BOute

Mélange (arXiv:2404.14527) and BOute (arXiv:2602.10729) optimize hardware allocation using Integer Linear Programming (ILP) and Bayesian Optimization based on query complexity. However, both frameworks are completely security-agnostic: when an attacker submits a semantically dense sponge prompt, these schedulers route the payload to high-capacity, expensive compute nodes. Our gateway introduces **Security-Aware Cost Co-Routing**, ensuring high-risk payloads are quarantined or sandboxed before reaching expensive backend clusters.

C. KV-Cache Protection vs. SGLang / RadixAttention

SGLang (arXiv:2312.07104) improves serving throughput by sharing KV-cache radix trees across requests with matching prefixes. However, shared prefix pools introduce a critical, unaddressed vulnerability: malicious prefixes can contaminate

the warm cache pool. Our gateway pre-filters malicious inputs before prefill execution, preventing adversarial contamination of shared KV-cache states.

VI. LIMITATIONS & FUTURE DIRECTIONS

- 1) **CPU Edge Proxy Scaling:** Under multi-gigabit ingress traffic, generating dense embeddings for thousands of concurrent requests per second on CPU proxy nodes may introduce CPU bottlenecking. Future work will investigate embedding distillation and ONNX runtime quantization (e.g., INT8 MiniLM) to compress latency further (< 1.5 ms).
- 2) **Security-Conditioned Heterogeneous ILP:** Integrating our continuous risk score directly into resource allocation algorithms (such as Mélange or BOute) will enable fine-grained scheduling across heterogeneous backend clusters.
- 3) **Preemptible Spot Instance Routing:** Low-risk requests can be safely assigned to spot instances, while Sandbox-tier queries are executed on isolated, time-capped on-demand nodes.

VII. CONCLUSION

We have presented **Adaptive LLM Security & Resource Risk Gateway**, an upfront CPU ingress proxy that neutralizes adversarial prompt injections and resource-exhaustion sponge attacks before backend model compute is allocated. By fusing BPE token count heuristics, repetition density analysis, TF-IDF n-grams, and calibrated MiniLM dense embeddings into a single multi-class XGBoost classifier, the gateway computes a unified risk score in under 5 milliseconds. Through a graduated execution tiering model (Full, Sandbox, Quarantine), the system achieves a **100% benchmark detection rate**, maintains a **false positive rate under 0.15%**, and delivers **55.6% to 82.4% cost savings**.

REFERENCES

- [1] G. Alon and M. Kamfonas, “Detecting language model attacks with perplexity,” *arXiv preprint arXiv:2308.14132*, 2023.
- [2] J. Dong, Z. Zhang, Q. Zhang, et al., “An engorgio prompt makes large language model babble on,” *arXiv preprint arXiv:2412.19394*, 2024.
- [3] C. Fernando, D. Banarse, et al., “Promptbreeder: Self-referential self-improvement via prompt evolution,” *arXiv preprint arXiv:2309.16797*, 2023.
- [4] Y. Jiang et al., “Heterogeneous GPU scheduling for LLM inference workloads,” *arXiv preprint arXiv:2502.00722*, 2025.

- [5] Manu, Y. Guo, J. Plested, T. Lynar, K. Thilakarathna, N. Sivaroopan, J. Yang, and W. Yang, “Prompt-induced over-generation as denial-of-service: A black-box attack-side benchmark,” *arXiv preprint arXiv:2512.23779*, 2025.
- [6] M. Phute, A. Helbling, et al., “LLM self defense: By self examination, LLMs know they are being tricked,” *arXiv preprint arXiv:2308.07308*, 2023.
- [7] N. Sivaroopan, K. Thilakarathna, A. Zomaya, Y. Guo, J. Plested, T. Lynar, J. Yang, and W. Yang, “SHIELD: An Auto-Healing Agentic Defense Framework for LLM Resource Exhaustion Attacks,” *arXiv preprint arXiv:2601.19174*, 2026.
- [8] I. Shumailov, Y. Zhao, D. Bates, N. Papernot, R. Mullins, and R. Anderson, “Sponge examples: Energy-latency attacks on neural networks,” in *IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 212–231, 2021.
- [9] L. Zheng, W. L. Chiang, Y. Sheng, et al., “SGLang: Efficient Execution of Structured LLM Programs with RadixAttention,” *arXiv preprint arXiv:2312.07104*, 2023.
- [10] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” *arXiv preprint arXiv:2307.15043*, 2023.